

Employee Task Management System

1. Project Overview

The Employee Task Management System is a web-based application designed to manage employee tasks efficiently within an organization.

This system allows Team Leads to:

- Create employee accounts
- Assign tasks to employees
- Track task progress
- Monitor task completion status

Employees can:

- Login securely
- View assigned tasks
- Update task status
- Manage task progress

The application improves communication between Team Leads and Employees and helps maintain organized task tracking.

2. Technologies Used

Frontend

- React.js
- React Router DOM
- Axios
- CSS
- SweetAlert2

Backend

- Node.js
- Express.js
- JWT Authentication
- bcryptjs

Database

- MongoDB Atlas
- Mongoose

3. System Architecture

Frontend (React.js)

↓

REST API (Express.js / Node.js)

↓

MongoDB Database

Flow:

1. User logs in
2. JWT token generated
3. Token stored in localStorage
4. Authorized requests sent to backend
5. Backend communicates with MongoDB
6. Response displayed in frontend

4. Modules

Team Lead Module

Features:

- Login
- Create Employees
- Assign Tasks
- View All Tasks
- Manage Employees

Employee Module

Features:

- Login
- View Assigned Tasks
- Update Task Status
- Dashboard Access

5. Authentication Flow

Team Lead Authentication

- Team Lead logs in using email/password
- JWT token generated
- Token stored in localStorage
- Protected routes enabled

Employee Authentication

- Employee logs in
- Employee token stored separately
- Access restricted based on role

6. Database Design

Employee Schema

```
{
  name: String,
  email: String,
  password: String,
  role: "employee"
}
```

Task Schema

```
{
  taskTitle: String,
  description: String,
  priority: String,
  status: String,
  assignedTo: ObjectId,
  createdAt: Date
}
```

7. Folder Structure

```
src/
├── Components/
│   ├── TeamLead/
│   ├── Employee/
│   └── Authentication/
├── Pages/
├── Routes/
├── CSS/
├── App.js
└── main.js

Backend:
backend/
├── controllers/
├── models/
├── routes/
├── middleware/
├── config/
└── server.js
```

8. API Endpoints

Authentication APIs

Method	Endpoint	Description
POST	/login	User Login
POST	/employee/create	Create Employee

Task APIs

Method	Endpoint	Description
POST	/task/assign	Assign Task
GET	/task/all	View Tasks
GET	/task/mytasks	Employee Tasks
PUT	/task/update/:id	Update Task

9. Assumptions Made

- Each employee has unique email
- Only Team Leads can assign tasks
- Employees can update only their own tasks
- JWT token expiration handled securely
- Internet connection required

10. Edge Cases Identified

Scenario	Handling
Invalid Login	Error message displayed
Empty Fields	Form validation
Unauthorized Access	Redirect to login
Expired Token	Logout user
Task Without Employee Validation	prevents submission

11. Issues Faced & Solutions

Issue 1: Token Authentication

Problem:

Protected routes accessible without login.

Solution:

Implemented JWT middleware validation.

Issue 2: Responsive Design

Problem:

Tables breaking on laptop/mobile screens.

Solution:

Used media queries and responsive CSS.

Issue 3: Task Update Delay

Problem:

Updated task status not reflecting instantly.

Solution:

Used React state updates and API refetching.

12. Security Features

- JWT Authentication
- Password Hashing using bcryptjs
- Protected Routes
- Role-based Access Control
- Input Validation